Minimum Operations to Sort a String

Problem

You are given a string **s** consisting of lowercase English letters. In one operation, you can select any substring of **s** that is **not** the entire string and sort it in non-descending alphabetical order. Return the minimum number of operations required to make **s** sorted in non-descending order. If it is not possible, return **-1**.

Examples

Example 1:
Input: s = "dog"
Output: 1
Explanation: Sort substring "og" to "go", making s = "dgo", which is sorted.

Example 2:
Input: s = "card"
Output: 2
Explanation: Sort "car" to "acr", then sort "rd" to "dr", making s = "acdr".

Example 3:
Input: s = "gf"
Output: -1
Explanation: An unsorted string of length 2 cannot be sorted because no proper substring contains both characters.

Approach

Sort a copy of the string to obtain the target non-decreasing order. If the original string already equals the target, return 0. If the string has length 2 and is unsorted, return -1, because its only proper substrings are single characters and cannot rearrange the two letters.

Otherwise, use the positions of the global minimum and maximum characters: if the first character is the minimum or the last character is the maximum, one operation sorts the remaining prefix or suffix. If either extreme appears in the middle, two operations suffice. If the minimum and maximum are at the wrong ends, three operations are required.

Java Solution

```
import java.util.Arrays;

class Solution {
    public int minOperations(String s) {
        int n = s.length();
        char[] sorted = s.toCharArray();
        Arrays.sort(sorted);

        if (s.equals(new String(sorted))) {
            return 0;
        }

        if (n == 2) {
            return -1;
        }
        char minimum = sorted[0];
```

```
            char maximum = sorted[n - 1];

            if (s.charAt(0) == minimum || s.charAt(n - 1) == maximum) {
                return 1;
            }

            for (int i = 1; i < n - 1; i++) {
                char c = s.charAt(i);

                if (c == minimum || c == maximum) {
                    return 2;
                }
            }

            return 3;
        }
    }
```

Complexity

Time complexity: O(n log n), due to sorting the string.
Space complexity: O(n) for the sorted copy.